

# Composable CXL Memory as a Kubernetes-Native Shared Memory for LLM Serving

A Feasibility Study on Seagate Composable Memory Appliance

Hongjian Fan, Kevin Zhang, David Habinsky, and Sean Dykstra  
*Research Group, Seagate Technology, LLC*

## Abstract

We present a Kubernetes Dynamic Resource Allocation (DRA) driver that makes composable CXL memory a schedulable cluster resource, and evaluate the resulting shared-memory tier for cross-node KV-cache reuse in LLM serving. The driver composes CXL regions on demand, materializes them as DAX devices on each participating host, and injects them into pods under a single Container Device Interface (CDI) name so that pods on different nodes access the same physical region. A shared-memory connector for vLLM/llm-d uses that region as a KV-cache tier with a slot directory embedded inside the shared medium, which eliminates the need for an external metadata service. On a two-node cluster with a 512 GiB CXL appliance and Qwen2.5-7B-Instruct, cross-node prefix reuse reduces TTFT by  $5.5\times$ – $36.6\times$  at an external hit rate of 95.4–99.5%, while node-local tiers (GPU prefix caching, CPU-DRAM offload) fall back to full recompute. The sharing gap, defined as the latency ratio between cross-node and same-node reuse, is 1–4%, indicating that cross-node reuse incurs little additional latency relative to same-node reuse on our testbed. Both replicas run full engines; the study demonstrates memory disaggregation rather than prefill/decode disaggregation. We report this as a feasibility study rather than a performance evaluation.

## 1 Introduction

The KV cache makes autoregressive serving fast and expensive. For Qwen2.5-7B-Instruct each attended token leaves behind 56 KiB of key and value tensors: a 32 K-token prompt is 1.75 GiB. Long shared prefixes, common in multi-turn chat, RAG, and agentic workflows, let later requests skip recomputation. Modern engines exploit this aggressively with in-GPU prefix caching [1, 2], host-DRAM and disk tiers [3, 4], and prefill/decode disaggregation [5, 6, 7].

What this needs is a cache tier that is both *large* and *shared*. The tiers available today provide one property or the other:

- **GPU VRAM**, fastest but smallest and confined to a single GPU.
- **Host DRAM**, an order of magnitude larger but confined to a single node.
- **RDMA and object tiers** [7, 8, 3], shared, but reuse traverses a transfer protocol and a NIC.

Compute Express Link (CXL) [9] offers a fourth option. A composable memory appliance can carve a region and map it to several hosts at once; each host sees byte-addressable memory at sub-microsecond latency. On our testbed it reads at 27–28 GB/s, which is  $2.4\times$  lower latency than a line-rate 100 GbE RoCEv2 fabric (Figure 1), with no transport layer on the reuse path: a KV block loads via a `memcpy` from a shared physical address.

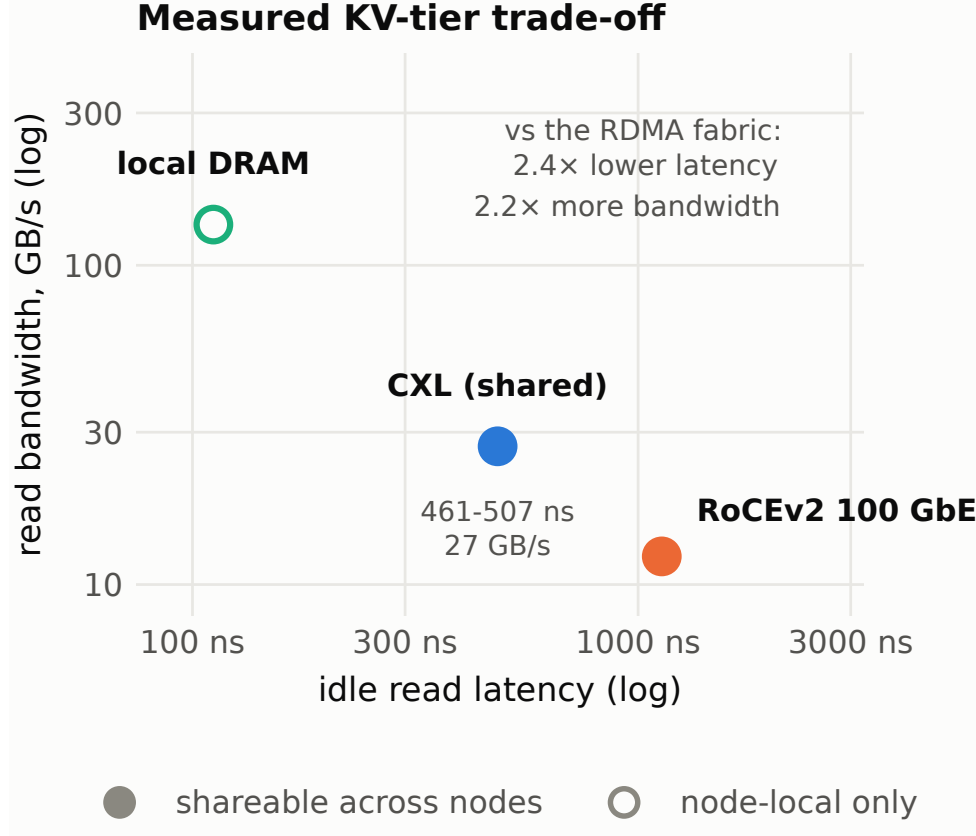


Figure 1: Measured trade-off between the candidate KV-reuse tiers on our testbed. Local DRAM is fastest but private; RoCEv2 is shared but protocol-bound. CXL is the only shareable tier within  $\sim 4.5\times$  of local DRAM latency, using ordinary loads and offering pooled capacity beyond per-node budgets.

**The gap.** What is missing is a *schedulable shared memory resource*. CXL appliances can compose regions visible to multiple hosts, yet Kubernetes lacks a resource type that captures that topology. The Pangaea v2 project [10] brings CXL memory into Kubernetes via an NRI plugin but only on a per-host basis. Recent CXL-KV systems [11, 12] assume the region already exists. Without a schedulable resource, shared CXL memory cannot be multiplexed between tenants or reclaimed on teardown.

**Background.** Transformer inference splits into a compute-bound *prefill* and a memory-bound *decode* [13]. Because the KV of a prefix depends only on that prefix, later requests with the same leading tokens can skip recomputation: the basis of prefix caching [1, 2]. Engines expose this as a tiering problem: vLLM’s `OffloadingConnector` abstracts a KV store behind a control-plane *manager* (which answers “do you have these blocks?” and reserves space) and per-worker *handlers* (which move tensors). Blocks are addressed by a content hash of the tokens they cover. Two properties matter for what follows: offloading has a *granularity* (256 tokens in our configuration), so any prefix not a multiple of it recomputes its trailing partial block; and the hash chain is seeded per process unless pinned, which is a prerequisite for cross-instance reuse (§4.3).

CXL [9] carries load/store semantics over PCIe electricals, letting a host address memory on a device. A *blade* in a *composable memory appliance* [14, 15] holds a pool of resource blocks and

several ports; a fabric manager carves a region and multi-assigns it to several hosts. On the host, a CXL device can be enumerated as a CPU-less NUMA node managed by the kernel page allocator, or as a `device_dax` character device that can be `mmap`ed for direct access, bypassing the page cache. Either path adds roughly  $2\text{--}5\times$  the latency of local DRAM [16, 17, 18]. We use `device_dax` so that the KV cache offload connector manages the memory space directly.

Kubernetes Dynamic Resource Allocation [19, 20] (`resource.k8s.io/v1` since v1.34) generalizes device plugins: a driver publishes `ResourceSlice` objects, a workload creates a `ResourceClaim`, the scheduler allocates, and a kubelet-side plugin *prepares* the claim, returning CDI [21] device names injected into the container. The model was designed for node-local accelerators (GPUs, NICs, FPGAs) and a `ResourceSlice` normally names the single node its devices are attached to. Composable memory breaks that assumption: the device does not exist until someone asks for it, and once composed it can be attached to several nodes at once.

**This paper.** We make composable CXL memory a schedulable Kubernetes resource and use it as a shared-memory tier for LLM serving (vLLM inside `llm-d` [22]). Our contributions:

1. **A Kubernetes DRA driver for composable CXL memory (§2).** It advertises blade capacity as a `ResourceSlice` whose `NodeSelector` spans every host cabled to the blade; composes a region on demand through a fabric manager; materializes it as a DAX sub-device on *each* participating host; and injects it under a single CDI name across all nodes. A finalizer and redundant teardown metadata make free-on-delete survive a plugin restart.
2. **An in-region metadata shared-memory KV tier (§3).** The directory resides inside the shared medium itself with an atomic slot bitmap plus a parallel key array, so presence lookup requires no external metadata service.
3. **An evaluation of shared memory for cross-node KV reuse (§4).** A two-node cluster, a 512 GiB appliance mapped to both hosts, and a cross-replica reuse experiment designed so that *only* a shared tier can produce a hit. We report the headline result, the *sharing gap*, together with the costs: the local premium, the achieved fraction of fabric bandwidth, and an assessment of data integrity under cross-node access.

**Scope.** This is a v1 feasibility report. The system is *not* prefill/decode disaggregated. Both replicas are full engines, so what we demonstrate is *memory* disaggregation. The harness is closed-loop and runs one session at a time, so time-to-first-token (TTFT) is the metric it supports; we make no goodput or tail-latency claim. Zero-copy GPU $\leftrightarrow$ CXL DMA, a pooled RDMA baseline, and a multi-tenant scheduling experiment are explicit non-goals. §6 enumerates the remaining limitations.

## 2 Scheduling Composable CXL Memory

Our driver, `cfm-dra-plugin.cxl.seagate.com`, runs as a `DaemonSet` on every CXL-capable node (Figure 2).

### 2.1 Resource model

The driver publishes **one `ResourceSlice` per blade**, containing a single device with attributes `applianceId` and `bladeId` and one capacity, `cxl_seagate_shared_mem_mib`, computed from the blade’s *unallocated* resource blocks.

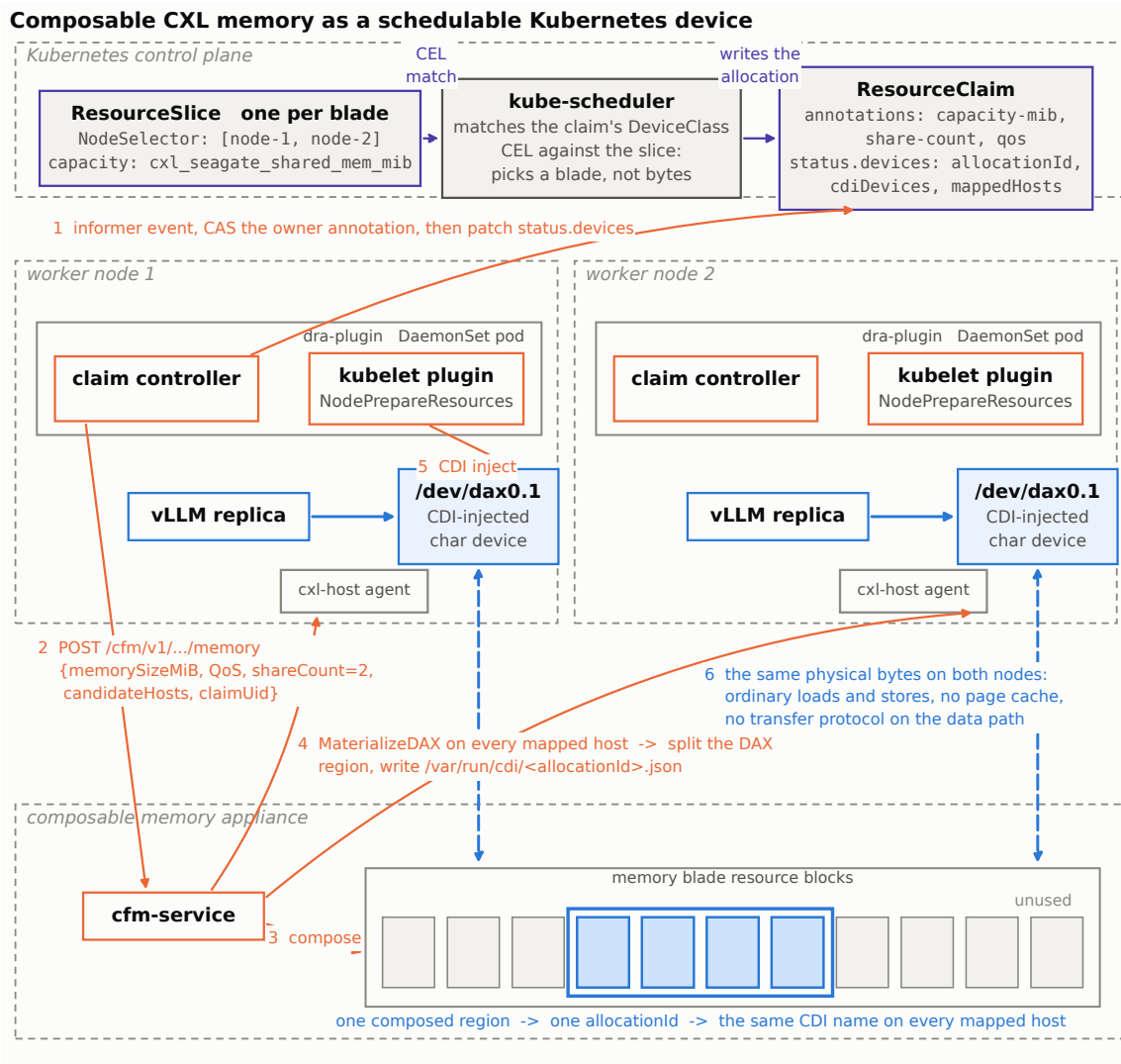


Figure 2: The DRA driver in context. Control-plane objects on the left: a **ResourceSlice** per blade whose node selector spans the connected hosts, and a **ResourceClaim** carrying size, share count and QoS in *annotations*. The **dra-plugin** **DaemonSet** on each node runs both a claim controller (one of which is elected to compose) and a kubelet plugin. **cfm-service** drives the blade over Redfish; **cxl-host** materializes the region locally. The result is one composed region reaching a pod on each node under a single CDI name.

The important field is the slice's node scope. A node-local driver sets **nodeName**; we set a **NodeSelector** matching `kubernetes.io/hostname` in [...] over exactly the hosts cabled to that blade. The scheduler may therefore place a consumer on any of them, since the region will be reachable from all. The cost is that the scheduler cannot express a locality preference finer than “connected” and that the driver, not the scheduler, decides which ports are assigned.

A cluster-scoped **DeviceClass** selects the driver with `device.driver == "cfm-dra-plugin.cxl.seagate.com"`; a workload's claim requests `deviceClassName: cfm-shared-memory`.

### ResourceClaim for a 512 GiB shared region

```
apiVersion: resource.k8s.io/v1
kind: ResourceClaim
metadata:
  name: cxl-kv-cache
  annotations:
    dra.cfm.seagate.com/capacity-mib: "524288"
    dra.cfm.seagate.com/share-count: "2"
    dra.cfm.seagate.com/qos: "4"
spec:
  devices:
    requests:
      - name: cxl-memory
        exactly:
          deviceClassName: cfm-shared-memory
```

*Size, share count and QoS are annotations — the scheduler picks a blade, not bytes.*

Figure 3: A claim for a 512 GiB region shared by two nodes. Size, share count and QoS are annotations, not part of the device request (§2.2). Both vLLM pods reference this one claim, so both access the same composed region.

## 2.2 Region sizing via annotations

A DRA `DeviceRequest` carries a device class and Common Express Language (CEL) selectors. It does not carry a quantity. Region size, share count and QoS class therefore travel as **annotations on the ResourceClaim** (`dra.cfm.seagate.com/capacity-mib`, `/share-count`, `/qos`), read by our controller after allocation. Figure 3 shows a complete claim.

The consequence is that the scheduler picks a *blade*, never a byte count. The advertised capacity is not decremented when a claim is allocated; it shrinks only on the next 30-second publish cycle. So there is no scheduler-side capacity accounting or overcommit protection: two claims created close together can be admitted against the same free capacity, and the second fails at compose time.

## 2.3 Compose-on-demand

Figure 4 traces the lifecycle. When the scheduler allocates the claim, every DaemonSet replica sees it:

1. **Wait for a consumer.** The controller waits (up to 30 s) for a pod referencing the claim to be bound to a node.
2. **Elect an owner.** Candidates race a compare-and-set on the `dra.cfm.seagate.com/owner` annotation; exactly one wins and calls the fabric manager. Standard DRA drivers need no election. They run as a single controller or work node-locally.
3. **Take a finalizer.** The winner adds `dra.cfm.seagate.com/free-memory` before composing anything, so a claim deleted mid-flight cannot vanish before blade capacity is released.
4. **Compose.** It reads capacity, share count and QoS from the annotations (Figure 3), builds the candidate host list from the blade’s port topology, and issues `ComposeMemory` to `cfm-service`. The blade composes a region from resource blocks matching the QoS class and *multi-assigns* it to each selected port.
5. **Materialize per host.** For each assigned port, `cfm-service` invokes a Redfish action on the connected node’s `cxl-host` daemon, which creates the local DAX device and writes a CDI spec (§2.5).

6. **Publish.** The controller patches the resulting CDI device names into `ResourceClaim.Status.Devices`, which kubelet reads.

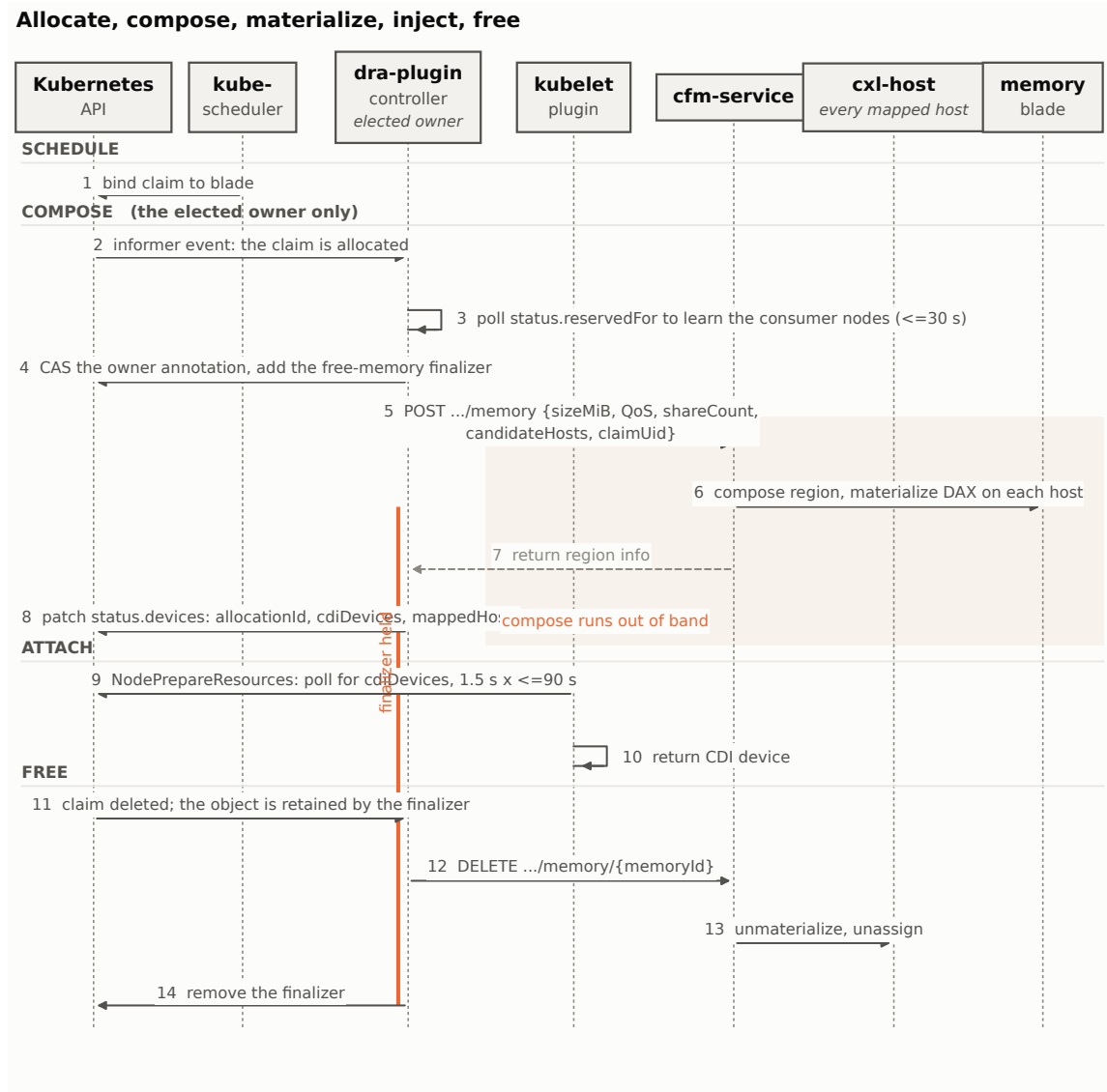


Figure 4: The allocate→compose→materialize→inject→free sequence. Note the owner election among DaemonSet replicas, the span over which the finalizer is held, and the shaded out-of-band window: the region is composed by a controller, not by the node that kubelet asks to prepare the claim, so `PrepareResourceClaims` has to wait for CDI names to appear in the claim status.

## 2.4 Single CDI name across nodes

The same region, identified by one `allocationId`, is materialized independently on each host, possibly at different DAX indices, since numbering is local. But the CDI device name is the allocation UUID, identical everywhere. A pod spec that names one claim therefore resolves, on whichever node it lands, to a node-local character device pointing at the composed region.

## 2.5 Host-side materialization and injection

`cxl-host` runs privileged on each CXL node and exposes a Redfish API to the fabric manager. On a materialize action it:

1. For CXL 1.1/2.0 devices, calls the **sysfs DAX split**: writing to `/sys/bus/dax/devices/` to carve a sub-device of the requested size, preceded if necessary by a pad device to align the offset. For CXL 3.x devices with DCD (dynamic capacity device) support, it requests capacity extension directly from the CXL kernel driver.
2. writes a CDI spec to `/var/run/cdi/<allocationId>.json` of kind `cxl.seagate.com/dax`, containing the device node to bind-mount plus the environment the workload consumes: `CXL_DAX_DEVICE` and `CXL_DAX_SIZE`.

Carving a sub-device per allocation gives per-allocation isolation inside a physically shared pool.

The kubelet plugin's `PrepareResourceClaims` reads CDI device IDs from the claim status: polling for up to 90s, because composition happens out of band relative to kubelet's request (the shaded window in Figure 4). `Unprepare` is a no-op: memory is released by the controller on claim deletion.

## 2.6 Lifecycle and teardown

The driver persists teardown metadata three ways: in memory, as a claim annotation, and in the claim's device status. It reconstructs free parameters from whichever survives. Combined with the finalizer, free-on-delete works across a plugin restart. If `FreeMemory` keeps failing, the controller retries five times and then removes the finalizer anyway, choosing a possible capacity leak over a claim that can never be deleted; the leak is visible in the fabric manager and recoverable out of band.

Two failure behaviors are deliberate. Materialization is *non-fatal*: if one host fails to create its DAX device, the compose succeeds and the pod on that node starts without a device. And if a blade is power-cycled, the fabric manager does not rediscover state automatically, because the current prototype does not persist composed-region metadata across power cycles. An operator must resynchronize after a power cycle.

The driver is Go against `resource.k8s.io/v1` and the `k8s.io/dynamic-resource-allocation` kubelet-plugin framework; it requires Kubernetes v1.34+ for GA DRA.

# 3 A Shared Memory Backend for LLM Serving

The connector is a storage backend behind `vLLM's OffloadingConnector`: a control-plane manager in the scheduler process answers lookups and reserves slots, and per-worker handlers move tensors. The device path and size come from the CDI environment injected in §2.5, so the same deployment manifest works on any node.

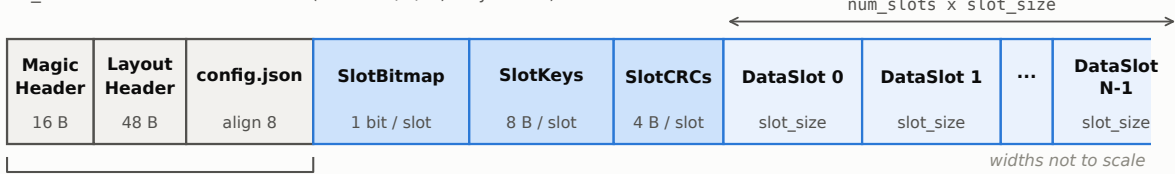
## 3.1 Mapping the region

A C++ engine opens the DAX device and `mmap`s it `MAP_SHARED`, then calls `madvise(MADV_RANDOM)` to suppress readahead. The optional `offset` argument enables multi-tenant partitioning: two independent pools can live in one device at different offsets. Because DAX has no page cache, the mapping *is* the device memory.

## In-region directory and data path of the CXL KV tier

### (a) A self-describing slot arena: the first writer formats it, later processes find the magic and attach

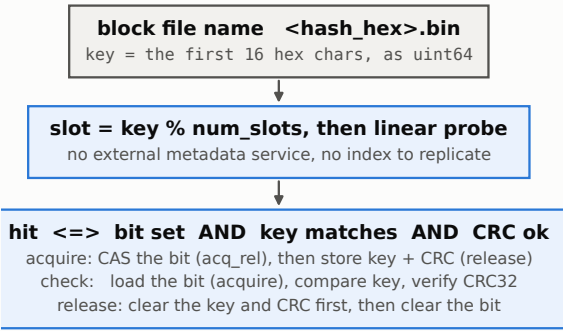
CXL\_MAGIC = 0x4C4C4D44584C0004 ("LLMDXL\0\4", layout v4)



CXL\_HEADER\_SIZE = 64 B, then the config JSON the first writer copied in

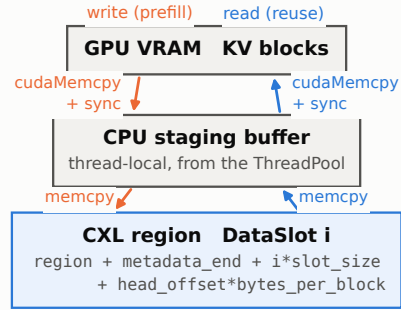
the in-region directory; it ends at metadata\_end. A GPU-less scheduler process answers a lookup from the header and these three structures alone -- it never maps a data slot.

### (b) Lookup: hash to a slot, then verify the key



The key array eliminates false hits from colliding hashes; the CRC32 array catches physical corruption of slot data.

### (c) Data path: host-staged, not zero-copy



DAX: hardware CXL.mem coherence, msync skipped, MADV\_RANDOM. File-backed fallback: msync(MS\_SYNC) on write, MS\_INVALIDATE on read.

Figure 5: The shared region. (a) The self-describing layout: magic and layout headers, the writer's configuration, the slot bitmap, the parallel key array, and the per-slot CRC32 array, then fixed-size data slots. (b) The lookup protocol: a slot key derived from the block-hash filename,  $\text{key} \% \text{num\_slots}$  with linear probing, and a hit only when the bit is set *and* the stored key matches, with the acquire/release ordering that makes it safe across processes. (c) The host-staged data path and the coherence rules for DAX versus file-backed mappings.

## 3.2 A self-describing region

Figure 5(a) shows the layout:

MagicHeader | LayoutHeader | config.json | SlotBitmap | SlotKeys | SlotCRCs | DataSlot...

The magic header ("LLMDXL" plus a version byte) signals whether the region is formatted. The layout header records slot size, slot count, and the end of the in-region directory area; the writer's config.json (model, dtype, block sizes) lets a later attacher validate compatibility. Slot size is derived from the engine's block geometry, and slot count from the region size.

The first process to map an unformatted region initializes it (headers, configuration, zeroed bitmap) and any later process *attaches* to the existing layout. There is no formatting step, no coordinator, and no ordering requirement between engines: whichever starts first formats, the other joins.

### 3.3 The directory lives inside the region

A shared key-value store requires a directory. Instead of an external service like Redis, etcd, or an RDMA-side registry, we put it in the shared medium itself (Figure 5(b)).

KV blocks are named by a content hash of the tokens they cover. The CXL layer takes the first 16 hex characters of vLLM’s offload-key basename as a 64-bit *slot key*. The slot index is `key % num_slots` with linear probing. Three arrays in the region implement the directory: a bitmap of occupancy (`atomic<uint64_t>` words), a parallel array of slot keys, and a per-slot CRC32 array. A writer claims a slot with `compare_exchange` on the bitmap word (acquire-release) and then publishes its key and CRC; a reader probes from the hash position and reports a hit only when the bit is set *and* the stored key matches *and* the CRC verifies, loaded with acquire ordering.

The key array is essential: an earlier bitmap-only checker reported a hit for every lookup once the region was non-empty, invalidating an entire dataset (§4.3). A bitmap alone is not a directory. The CRC32 array adds physical integrity: it catches bit flips, partial writes, and coherence edge cases that a key match alone cannot detect.

Because the directory is just three arrays at known offsets, the GPU-less scheduler process answers lookups by `mmap` ing the region from Python and reading them. No metadata service exists to deploy, scale, or lose.

Two limitations follow. The slot key is derived from the block-hash basename only, so the region has no model discriminator: two models sharing a region would collide. And the key is verified before the copy but not re-verified after, so a release-and-reacquire could in principle alias (§6).

### 3.4 The data path, and what it costs

Figure 5(c) shows both directions. A store copies GPU blocks into a thread-local host staging buffer, synchronizes the CUDA stream, acquires a slot, `memcpy` s to `region + metadata_end + slot_idx*slot_size`, and issues a sequentially-consistent fence. A load probes for the slot, `memcpy` s from the region into staging, and copies staging to the GPU.

This is **host-staged, not zero-copy**: the GPU never DMAs to or from CXL. §4.6 quantifies the cost: we achieve 5.8 GB/s against a 27 GB/s fabric, leaving 3–5× headroom unrealized. Zero-copy will require future CXL devices with peer-to-peer (P2P) support.

We also have no eviction. Slots are effectively write-once: no TTL, no LRU, no background reclaim. A store that finds no free slot fails softly.

### 3.5 Coherence

Correctness across nodes rests on hardware CXL.mem coherence over the DAX mapping. We flag the DAX case as a threat to validity rather than a proven property. Our experiments provide strong *behavioral* evidence: tens of thousands of cross-node hits, a zero-false-positive control run, a same-versus-cross latency difference of 1–4%, and zero CRC32 mismatches over 1.2 TB of writes (§4.9), but we did not verify loaded bytes against written bytes directly, and §4.9 explains why our attempt was inconclusive.

### 3.6 Implementation notes

The KV tier is a C++ engine with Python glue behind vLLM’s `OffloadingConnector`, running inside vLLM v0.23.0. During development, a seeding issue invalidated an early dataset: vLLM 0.23 seeds its block-hash chain from `os.urandom()` when `PYTHONHASHSEED` is unset. Two engines therefore derive *different* hashes for identical tokens, and every cross-instance lookup misses silently, with

Table 1: Testbed and measured tier characterization. Latency and bandwidth are measured with Intel MLC on both worker nodes.

| Tier                                   | Latency                                          | Bandwidth      |
|----------------------------------------|--------------------------------------------------|----------------|
| Local DRAM (NUMA 0)                    | 111 ns                                           | 134 GB/s       |
| CXL appliance (512 GiB, dual-attached) | 461–507 ns                                       | 27.1–28.5 GB/s |
| RoCEv2 (100 GbE, direct-connect)       | 1.13 $\mu$ s                                     | 12.25 GB/s     |
| Worker nodes                           | 2 $\times$ EPYC 9254, 128 GB DDR5, 1 $\times$ L4 |                |
| Software                               | K8s v1.35.7, vLLM v0.23.0, 11m-d                 |                |
| Model                                  | Qwen2.5-7B-Instruct, bf16, 56 KiB/token          |                |

a plausible-looking zero hit rate. `PYTHONHASHSEED` must be set to the same value on every replica sharing a region.

## 4 Evaluating Shared Memory for Cross-Node KV Reuse

All values in this section are computed by our analysis script from the raw per-request logs, and every table and figure is generated from the same `stats.json` ; nothing is transcribed by hand.

### 4.1 Testbed

**Compute nodes.** Two worker nodes, each a single-socket AMD EPYC 9254 (24c/48t), 128 GB DDR5, one NVIDIA L4 (24 GB), running Kubernetes v1.35.7. The nodes are directly connected by 100 GbE ConnectX-6 Dx RoCEv2, characterized in §4.2.

**CXL appliance.** An FPGA-based prototype memory blade for the Seagate Composable Memory Appliance. It provides two host ports, each running CXL 1.1 over PCIe Gen5 x16, and holds 1 TiB of composable resource backed by 8 $\times$ 128 GiB DDR4 modules at 1866 MHz in 2DPC. The two-port topology yields a two-node cluster: each port connects to one host, and a composed region is multi-assigned to both. We compose 512 GiB, rather than the full 1 TiB, to avoid an AMD BIOS memory hole at the 1 TiB address boundary. A production appliance with more ports would let the same driver serve four or more consumers on a single blade.

**Software and model.** Serving is vLLM v0.23.0 with 11m-d , one replica per node at tensor parallelism 1. The model is Qwen2.5-7B-Instruct [23] (bf16, 56 KiB of KV per token).

**GPU capacity.** A single NVIDIA L4 (24 GB VRAM) per node is sufficient for closed-loop, one-session-at-a-time prefill measurements but prevents us from running concurrent decode traffic alongside the reusing request: a 32K-token prefill plus active decode slots would exceed VRAM capacity. Consequently we cannot report goodput, tail latency under load, or contention behavior: §6 discusses the implications.

Table 1 collects the testbed and its measured characterization.

### 4.2 What the fabric can do

Memory Latency Checker [24] measurements on both nodes yield: local DRAM at 111 ns / 134 GB/s; the CXL appliance at 461–507 ns / 27.1–28.5 GB/s; and the RoCEv2 link at 1.13  $\mu$ s

Table 2: The four KV-reuse configurations under comparison. Only T4 places KV blocks somewhere a replica on the other node can reach.

| Tier | KV reuse mechanism              | vLLM configuration                                                                                                                             | Shared?    |
|------|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| T0   | none (recompute floor)          | <code>--no-enable-prefix-caching</code>                                                                                                        | —          |
| T1   | GPU VRAM prefix cache           | <code>--enable-prefix-caching</code>                                                                                                           | no         |
| T2   | CPU DRAM offload                | <code>--no-enable-prefix-caching</code><br><code>--kv-offloading-backend=native</code><br><code>--kv_offloading_size=32</code>                 | no         |
| T4   | <b>shared CXL region (ours)</b> | <code>OffloadingConnector,</code><br><code>backend=CXL,</code><br><code>cxl_mode=read_write,</code><br><code>--no-enable-prefix-caching</code> | <b>yes</b> |

/ 12.25 GB/s (98 % of line rate). These numbers are consistent with prior CXL performance studies on data workloads [25] and shared-memory access benchmarks [26]. On this hardware CXL is  $2.4\times$  lower latency and  $2.2\times$  higher bandwidth than the RDMA fabric, and byte-addressable: a KV block is read with ordinary loads rather than a transfer protocol.

### 4.3 Methodology

**What we measure.** Client-observed TTFT over SSE streaming, from the first byte of the request to the first generated token. Hit rates come from vLLM’s `external_prefix_cache_{queries,hits}` counters, scraped on **both** replicas before and after **each** request.

**The experiment.** Each session issues two requests sharing a long prefix:

```
req1 = SHARED + Q1  → replica A  (writes KV into the tier)
req2 = SHARED + Q2  → replica A (same) | replica B (cross)
```

The **cross** arm isolates the shared-memory effect: replica B never saw those tokens, and node-local tiers (GPU prefix caching, CPU-DRAM offload) cannot serve it. We drive the replicas directly, so placement is deterministic. The cross arm also avoids a short-circuit in the connector’s manager, which records locally stored keys and answers same-replica lookups without consulting the region.

**Grid.** 4 tiers (Table 2)  $\times$  2 arms  $\times$  4 prefix lengths  $\times$  20 sessions  $\times$  3 repetitions = 1920 request pairs. We label the CXL tier T4 rather than T3 to reserve T3 for an RDMA-based tier in an extended evaluation. Prefix lengths are tokenizer-calibrated; the nominal 2 K/8 K/16 K/32 K buckets are medians of 2 054 / 8 173 / 16 282 / 32 694 tokens. Body text is generated per session from a seeded template grammar. Tier order is varied per repetition so that ordering and thermal drift cannot masquerade as a tier effect, and the CXL region is wiped between repetitions.

**Prerequisites.** `PYTHONHASHSEED=0` on both replicas (§3), and an offload granularity of 256 tokens, so any prefix not a multiple of 256 recomputes its trailing partial block on every tier.

**Validity controls.** Three checks run on every cell.

1. **No prompt leakage.** `req1` hit rate must be  $\approx 0$ . Measured:  $\leq 0.2\%$  on every tier; 32 of 32 cells pass a 5 % threshold.

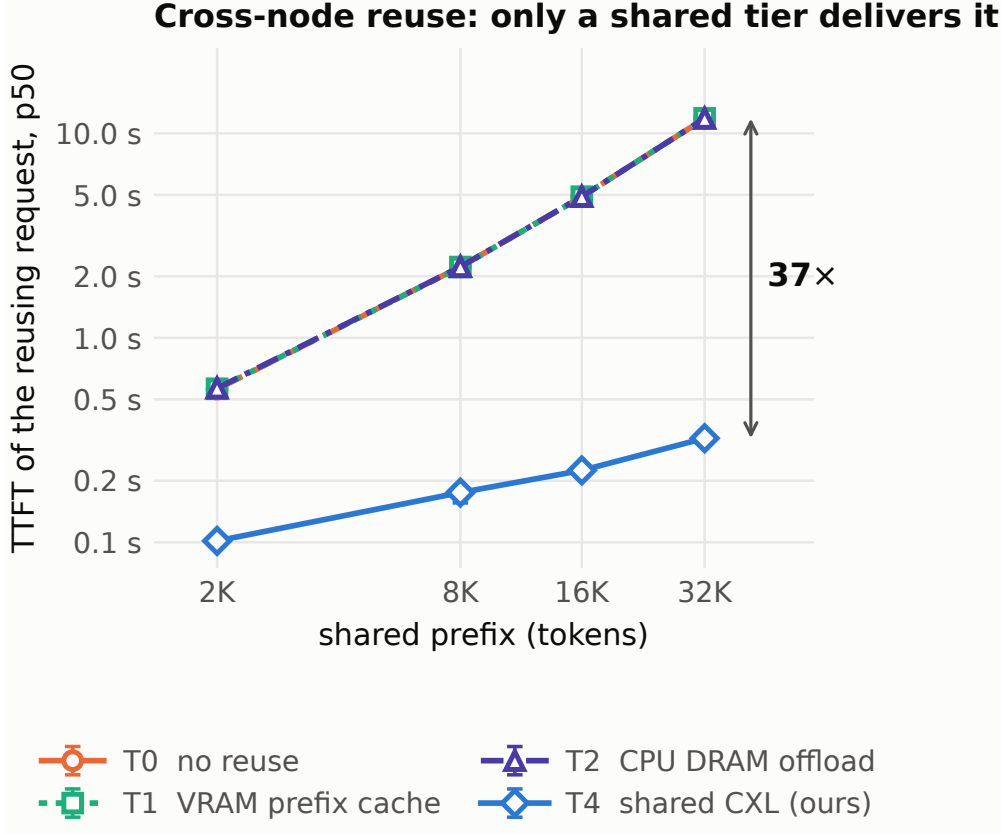


Figure 6: Cross-node reuse. When the reusing request lands on the other node, the GPU prefix cache (T1) and CPU-DRAM offload (T2) coincide with full recompute (T0): neither cache exists on the node that must serve the request. The shared CXL tier (T4) serves it, and its advantage grows with context length. Error bars span the three repetitions.

2. **No false hits.** A separate control run confirms **0 hits out of 12 298 queries** for never-stored keys, ruling out the bitmap-only defect of §3.
3. **Fresh region.** The harness aborts a T4 run unless exactly one replica logs an initialize and the other a reuse.

**Bounding the same-versus-cross bias.** The two nodes are not perfectly matched, and each arm has its own prompt set. T0 measures both at once: its cross-arm median TTFT is 1.4%, 3.6%, 4.5%, and 4.6% *below* its same-arm median at 2 K, 8 K, 16 K, and 32 K. Because the cross arm is naturally faster (by up to  $\sim 5\%$ ), the collapse of T1/T2 in the cross arm is conservative: had the bias gone the other way, the penalty would appear smaller. So **every cross/same ratio carries a combined systematic bias of  $\leq 5\%$** . Tier-to-tier comparisons within the cross arm are unaffected.

#### 4.4 Only a shared tier delivers cross-node reuse

Figure 6 and Table 3 give the primary result (raw TTFT percentiles in Table 6). In the cross arm, T1 and T2 are indistinguishable from having no cache at all; the shared region turns the same request into a load, yielding a 5.5–36.6 $\times$  speedup over recompute (median TTFT,  $n = 60$  per cell).

Table 3: Cross-node KV reuse. Speedups are ratios of median TTFT; *cross/same* is 1.0 for a tier that shares perfectly and grows without bound for a node-local one.

| Prefix | T0               | T1      | T2      | T4           | T4 vs T0 | T4 hit | cross/same |       |
|--------|------------------|---------|---------|--------------|----------|--------|------------|-------|
|        | median TTFT (ms) |         |         |              |          |        | T1         | T4    |
| 2K     | 560.7            | 563.3   | 566.8   | <b>101.6</b> | 5.5×     | 95.4%  | 6.5×       | 1.02× |
| 8K     | 2223.6           | 2218.6  | 2224.6  | <b>175.0</b> | 12.7×    | 98.0%  | 19.0×      | 1.04× |
| 16K    | 4895.0           | 4898.4  | 4901.9  | <b>224.6</b> | 21.8×    | 99.0%  | 37.2×      | 1.04× |
| 32K    | 11816.9          | 11845.0 | 11822.3 | <b>322.6</b> | 36.6×    | 99.5%  | 62.6×      | 1.01× |

The sharing gap: node-local tiers lose the hit when the request moves

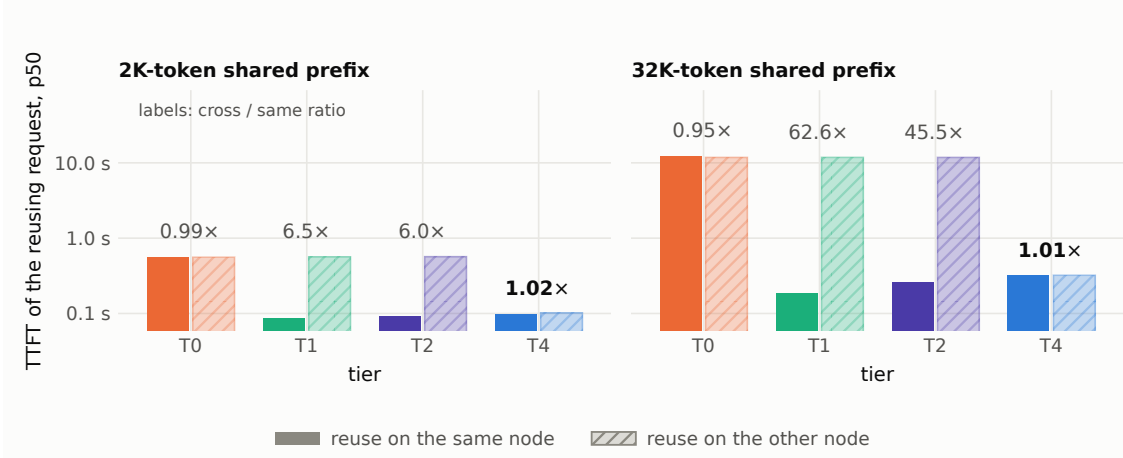


Figure 7: The sharing gap: the same data read as a within-tier question. What does it cost a tier when the reusing request moves to the other node? For the node-local tiers the hit is simply lost; for the shared region there is no boundary to cross.

The speedup grows with prefix length because the two sides scale differently. Recompute is superlinear in context, with an effective prefill rate that falls from 3 657 to 2 767 tok/s between 2 K and 32 K, while the CXL read path is dominated by data movement. A least-squares fit gives

$$\text{TTFT} \approx 104 \text{ ms} + 6.90 \mu\text{s}/\text{token} \quad (1)$$

with residuals  $< 17$  ms.

Reuse has a break-even prefix length below which the tier’s fixed cost is not worth paying. Extrapolating the two trends puts it in the *low hundreds of tokens*.

## 4.5 The sharing gap

Figure 7 asks the within-tier question: *what does it cost a tier when the reusing request moves?*

Table 4 summarizes the ratio (raw medians from Table 6). T1 and T2 lose their hit entirely when the request moves: a 6–63× penalty growing with context. **T4 is within 1–4 % of its own same-node number at every prefix length.** A 36× speedup quantifies the recompute penalty; a 1.02× cross/same ratio measures the architecture.

Table 4: The sharing gap: cross-arm median TTFT divided by same-arm median TTFT. A value of 1.0 means the tier shares perfectly; node-local tiers lose their hit entirely when the request moves.

| Prefix | T0    | T1    | T2    | T4           |
|--------|-------|-------|-------|--------------|
| 2K     | 0.99× | 6.5×  | 6.0×  | <b>1.02×</b> |
| 8K     | 0.96× | 19.0× | 16.1× | <b>1.04×</b> |
| 16K    | 0.95× | 37.2× | 28.8× | <b>1.04×</b> |
| 32K    | 0.95× | 62.6× | 45.5× | <b>1.01×</b> |

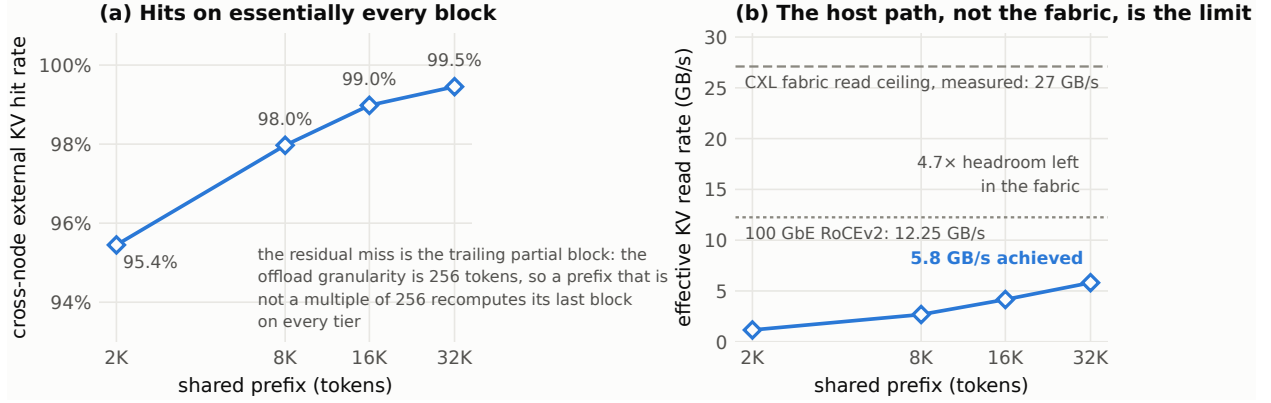


Figure 8: (a) Cross-node external hit rate against the 256-token block-alignment bound: every fully aligned block hits, and the residual is one trailing partial block. (b) Achieved effective read rate against the measured fabric ceiling: the limit we measured is our host-staged copy, not CXL.

#### 4.6 Where the hits and the time go

**Hit rate.** Cross-node external hit rate is 95.4% / 98.0% / 99.0% / 99.5% at 2K/8K/16K/32K (Figure 8(a)). This is the **block-alignment bound**: the offload granularity is 256 tokens, the trailing partial block is recomputed on every tier, and measured hits track the aligned fraction to within 0.9 pp at  $\geq 8K$  (and 0.05 pp at  $\geq 16K$ ). Every fully aligned block hits.

**Where the time goes.** Equation 1 decomposes the cross-arm TTFT into a fixed component and a per-token marginal cost:

- **~104 ms fixed.** Most of this is engine overhead, not CXL: T1 (a pure VRAM hit, same arm) costs 87 ms at 2K, so the CXL path adds only ~13 ms on top of the vLLM floor (CUDA graph setup, scheduler dispatch, Python glue). The intercept is not a CXL tax: it is the cost of running the engine.
- **6.90  $\mu$ s/token marginal,** i.e. 8.3 GB/s on the data path, or 31% of the 27 GB/s fabric bandwidth limit.

The bottleneck is the host-staged data path: KV is copied CXL  $\rightarrow$  CPU staging  $\rightarrow$  GPU (two memcpy hops) rather than DMA'd. The measured rate is limited by this implementation choice; a zero-copy path offers 3.3x headroom on the marginal rate.

Dividing the reloaded KV volume by the full TTFT (fixed + marginal) yields a lower effective rate because the ~104 ms fixed cost amortizes over the transfer. At 32K tokens the effective rate is 5.81 GB/s (21% of the limit; Table 5, Figure 8(b)), rising toward the marginal 8.3 GB/s as prefix length grows. At short prefixes the effective rate is misleadingly low (1.16 GB/s at 2K), and the

Table 5: Effective KV read rate achieved by the shared tier, computed as the reloaded KV volume divided by the whole median TTFT (so it includes the engine’s fixed cost). The fabric ceiling is the 27.1 GB/s measured in Table 1.

| Prefix | KV volume | T4 p50 TTFT | effective read | % of ceiling |
|--------|-----------|-------------|----------------|--------------|
| 2K     | 112 MiB   | 101.6 ms    | 1.16 GB/s      | 4%           |
| 8K     | 447 MiB   | 175.0 ms    | 2.68 GB/s      | 10%          |
| 16K    | 891 MiB   | 224.6 ms    | 4.16 GB/s      | 15%          |
| 32K    | 1788 MiB  | 322.6 ms    | 5.81 GB/s      | 21%          |

**Distribution of cross-node reuse TTFT (n=60 per tier per panel, 3 reps)**

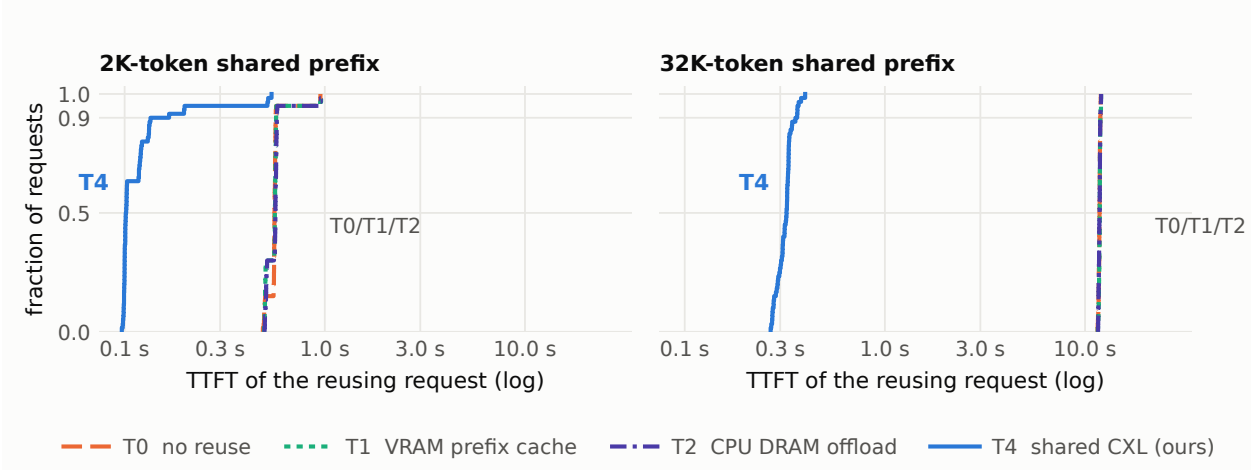


Figure 9: TTFT distribution in the cross arm at the shortest and longest prefix ( $n = 60$  per tier per panel). Distributions are tight for every tier; this is a closed-loop, one-session-at-a-time harness, so no tail-latency-under-load claim can be made from it. T4’s only visible tail is the single first-touch request discussed in the text.

fabric is idle during the engine’s fixed overhead, not saturated during the copy.

#### 4.7 TTFT distribution and first-touch warm-up

TTFT distributions are tight for every tier (Figure 9, Table 6). At  $\geq 8K$  tokens T4’s p99/p50 is 1.11–1.18, comparable to T0’s 1.01–1.03 while sitting an order of magnitude lower in absolute terms.

T4’s only tail is at 2 K tokens (p99/p50 = 5.13), and it is one request: the reading replica’s first CXL read costs **~420 ms extra** (521 ms vs. 101 ms steady state) for DAX first-touch page faults and staging-buffer allocation. It is paid once per process: the 8 K, 16 K and 32 K cells that follow show no spike (Figure 10). In the whole cross arm exactly **3 of 240 requests** exceed 450 ms, one per repetition, all first reads.

#### 4.8 Cross-node availability beyond VRAM capacity

When reuse *is* local, VRAM prefix caching still wins: by  $1.15\times$ – $1.70\times$  (Table 7): because no remote tier can match the bandwidth of memory on the same die as the compute.

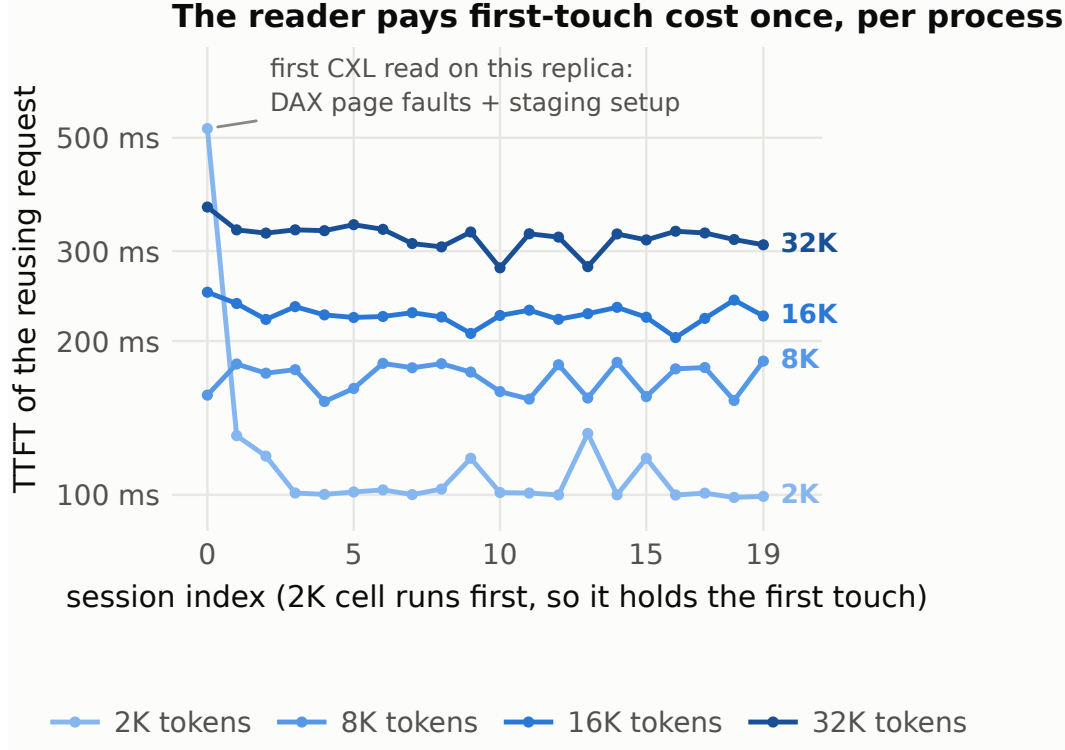


Figure 10: The reading replica pays a  $\sim 420$  ms first-touch cost once per process, from DAX page faults and staging-buffer allocation, and never again. The 2K cell runs first and absorbs it; the longer cells that follow show no spike.

A shared CXL region is visible to every attached host simultaneously and persists independently of per-replica VRAM pressure. Consider a popular prefix evicted from one GPU’s prefix cache under memory pressure: the CXL tier still serves it to any replica that needs it, avoiding a full re-prefill. The tier complements prefix caching by serving requests when the per-GPU budget is exceeded or the request has been load-balanced to a different pod. The trade:  $\leq 1.7\times$  **premium when reuse is local**, against a **6–63 $\times$  penalty when it is not**.

#### 4.9 Correctness: end-to-end integrity with bf16 rounding

Cross-node integrity checking relies on three layers: vLLM’s content-addressable block hash (same tokens produce the same block identity), the per-slot key array (eliminates false-positive reads from hash collisions), and a per-slot CRC32 checksum over the exact byte range written (catches physical corruption). A full cross-replica benchmark reports **zero CRC32 mismatches** over approximately 1.2 TB of CXL writes, with 95–99 % external hit rate on the reading pod. The cross-replica TTFT matches the same-replica TTFT to within 1–4 % (Table 4); corrupted data would trigger cache misses and prefill fallback, making the cross arm significantly slower.

Replaying 10 identical prompts through T0 and T4 (greedy decoding, `temperature=0`, `seed=0`) yields **6/10 byte-identical continuations**. The 4 divergences are caused by bf16 rounding: near-tie values may round differently across GPUs, producing at most 1 ULP difference in the mantissa. The CRC32 protects against *corruption*, not *divergence*: a legitimate rounding difference produces a valid checksum. Because the KV cache is an intermediate representation, a 1 ULP difference in a bf16 attention score has negligible impact on generation quality in practice.

#### 4.10 Reproducibility

Three full repetitions in different tier orders. Per-repetition median TTFT (Table 8) agrees to within 3.1 % on 15 of 16 cross-arm cells; the exception is T4 at 8 K tokens, where the three medians span 12.9 % (156 / 174 / 178 ms), still an order of magnitude inside the effect being measured.

The remaining limitations are collected in §6.

### 5 Related Work

KV-cache reuse has moved from single-server prefix caching to cross-node pooled storage. vLLM [1] introduced PagedAttention, which virtualizes KV memory inside a single GPU; SGLang [2] added aggressive prefix reuse. Systems like LMCache [3] and CacheBlend [4] extend the cache to host DRAM and remote object storage, while Mooncake [7] pools KV across nodes via RDMA. Three recent systems target CXL specifically: TraCT [11] demonstrates rack-scale CXL KV transfer for disaggregated serving, SAC [27] targets sparse-attention workloads, and HyMCache [12] proposes a CXL memory rack for multi-turn serving.

Table 9 contrasts these systems with our approach. TraCT, SAC, and HyMCache assume the CXL region already exists and focus on the KV storage layer. Our DRA driver makes the region itself schedulable by Kubernetes. Mooncake achieves pooled storage via RDMA but requires a transfer protocol on every access; our CXL tier uses byte-addressable loads with no transport layer. None of these systems integrate with Kubernetes scheduling beyond static node assignment.

Prefill/decode disaggregation [5, 6] separates phases to improve accelerator utilization but does not address KV sharing across replicas. Our memory disaggregation is complementary and can underlie a future P/D handoff.

The broader memory disaggregation literature, including FaRM [29], Infiniswap [30], and GAM [31], exports memory through explicit transport protocols, typically RDMA. Composable CXL memory exposes shared memory through load/store semantics, so applications access it with ordinary loads and stores. CXL pooling systems like Pond [32] and TPP [16] study cloud-scale pooling and OS-level page placement but do not target Kubernetes scheduling. Octopus [33] explores sparse topology for CXL memory pods to improve scalability. Wang et al. [18] survey programming and optimization techniques for cache-coherent heterogeneous interconnects including CXL, NVLink-C2C, and AMD Infinity Fabric. Weisgut et al. [25] study CXL memory performance for in-memory data processing, and CXL-Bench [26] provides a benchmarking framework for shared CXL memory access. PolarCXLMem [28] demonstrates CXL switch-based disaggregated memory for cloud-native databases, showing up to  $2.1\times$  throughput improvement over RDMA in pooling scenarios.

Kubernetes Dynamic Resource Allocation [19, 20] generalized the device plugin model with DeviceClasses, ResourceSlices, and ResourceClaims. Existing DRA drivers target node-local resources (GPUs, FPGAs, networking). Pangaea v2 [10] brings CXL memory into Kubernetes but only on a per-node basis. We are unaware of any DRA-managed memory resource that is dynamically composed and simultaneously attached to multiple nodes.

### 6 Limitations and Threats to Validity

**Not P/D-disaggregated.** Both replicas are full `kv_both` engines; we demonstrate memory disaggregation. A real P/D handoff over the shared region is the primary v2 deliverable.

**No pooled-RDMA baseline.** We characterize the RoCEv2 fabric but do not run a hash-keyed RDMA KV pool. A pooled RDMA baseline is a v2 deliverable.

**Single GPU, no concurrent load.** Each node has one NVIDIA L4 (24 GB VRAM). A 32 K-token prefill plus active decode slots exceeds VRAM capacity, so we cannot run background traffic during measurements. The results are isolated-prefill TTFT; they do not bound goodput, tail latency under queueing, or contention between prefill and decode. On larger GPUs (A100, H100) the tier’s behavior under load would be the more revealing test: the CXL read competes with decode for the PCIe/GPU memory bus.

**Two-node evaluation.** The appliance prototype has two host ports, yielding a two-node cluster (§4). The DRA driver is not two-node-specific: it enumerates ports from blade topology and composes regions for any share count. Evaluating four or more consumers on a single blade requires a production appliance with more ports.

**Narrow workload.** One model, one size, 100 % prefix reuse by construction, no reuse-ratio sweep, no capacity pressure, and no multi-tenant scheduling experiment. The latter being where the DRA driver’s value should be measured.

**Host-staged data path.** 5.8 GB/s achieved against a 27 GB/s fabric; 3–5× headroom unrealized without GPUDirect-style DMA.

**Correctness: bf16 rounding, not corruption.** Cross-node data integrity is verified by per-slot CRC32 (zero mismatches over 1.2 TB of writes). Six of ten prompts produce byte-identical continuations (§4.9); the remaining four differ by at most 1 ULP due to bf16 rounding across GPUs. Residual hazards: the slot key is not re-verified after the copy (alias possible on release-and-reacquire), and the store path silently discards write failures (invisible to cross-arm results but corrosive under capacity pressure).

**Coherence is assumed.** Hardware CXL.mem coherence over the DAX mapping is assumed; evidence is behavioral, not architectural.

**No eviction.** Write-once slots, no TTL, no reclaim: not a production tier.

**No scheduler-side capacity accounting.** Size travels in annotations; advertised capacity refreshes on a 30-second cycle; concurrent claims can race the same free capacity.

**Imperfectly matched nodes.** Bounded at  $\leq 5\%$  on cold prefill (T0), conservative in direction for every reported ratio.

## 7 Conclusion and Future Work

This work demonstrates that dynamically composed, multi-host CXL memory can be exposed as a first-class Kubernetes DRA resource and used as a shared KV-cache tier for LLM serving. We presented a DRA driver that makes composable CXL regions schedulable by Kubernetes: the driver advertises blade capacity, composes regions on demand, attaches them to multiple nodes, isolates allocations via DAX sub-devices, and releases capacity on claim deletion. We also presented a KV tier for vLLM/llm-d that embeds its directory inside the shared region, which removes the dependency on an external metadata service. On a two-node testbed with a 512 GiB CXL appliance, cross-node prefix reuse reduced TTFT by 5.5–36.6× at a hit rate bounded only by block alignment, while node-local tiers fell back to full recompute. The sharing gap was 1.01–1.04× for the shared region compared to 6.5–62.6× for node-local tiers when the reusing request moved nodes.

VRAM prefix caching remains faster by up to 1.7× when reuse is local; the CXL tier provides availability when the request lands on a different node. The remaining limitations are in §6.

Future work includes: (i) prefill/decode disaggregation with handoff through the shared region; (ii) a pooled RDMA baseline comparable to Mooncake; (iii) multi-tenant scheduling under con-

tention with scheduler-side capacity accounting; and *(iv)* eviction and post-copy key re-verification to close the remaining alias window.

## Artifact Availability

The DRA driver, CXL KV connector, benchmark harness, raw per-request logs for all three repetitions, and the scripts that generate every table and figure in §4 will be released on the Seagate GitHub repository [34]. Regenerating the evaluation from raw logs is three commands; no number in this paper is transcribed by hand.

## References

- [1] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proc. SOSP*, 2023.
- [2] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng. SGLang: Efficient execution of structured language model programs. In *Proc. NeurIPS*, 2024.
- [3] Y. Liu, Y. Cheng, J. Yao, Y. An, X. Chen, S. Feng, Y. Huang, S. Shen, R. Zhang, K. Du, and J. Jiang. LMCahe: An Efficient KV Cache Layer for Enterprise-Scale LLM Inference. arXiv preprint arXiv:2510.09665, 2025.
- [4] J. Yao, H. Li, Y. Liu, S. Ray, Y. Cheng, Q. Zhang, K. Du, S. Lu, and J. Jiang. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. In *Proc. EPOS*, pp. 94–109, 2025.
- [5] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *Proc. USENIX OSDI*, 2024.
- [6] P. Patel, E. Choukse, C. Zhang, A. Shah, Í. Goiri, S. Maleki, and R. Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. In *Proc. ISCA*, 2024.
- [7] R. Qin, Z. Li, W. He, J. Cui, F. Ren, M. Zhang, Y. Wu, W. Zheng, and X. Xu. Mooncake: Trading more storage for less computation — a KVCache-centric architecture for serving LLM chatbot. In *Proc. USENIX FAST*, pp. 155–170, 2025.
- [8] NVIDIA. NIXL: NVIDIA Inference Xfer Library. <https://github.com/ai-dynamo/nixl>.
- [9] CXL Consortium. *Compute Express Link Specification*, Revision 3.1, 2023.
- [10] H. D. Lee, J. Park, Y. Lee, J. Im, J. Jung, J. So, S. Tavallaei, T. Lee, W. T. Shim, C.-H. Chang, J. Jiang, S. Ryu, T. Song, W. Shin, and S. Hwang. Pangaea v2: CXL-Based Disaggregated Memory System Architecture for Cloud-Native Orchestration. *IEEE Transactions on Computers*, vol. 75, no. 4, pp. 1261–1275, 2026.
- [11] D. Yoon, Y. Min, H. Kim, S. H. Noh, and J. Kim. TraCT: Disaggregated LLM Serving with CXL Shared Memory KV Cache at Rack-Scale. arXiv preprint arXiv:2512.18194, 2025.
- [12] H. Jang, I. Song, S. H. Noh, and J. Kim. HyMCache: A KV Cache Framework for Multi-Turn LLM Serving with CXL-Hybrid Memory. arXiv preprint arXiv:2607.18141, 2026.
- [13] J. Pan and G. Li. A Survey of LLM Inference Systems. arXiv preprint arXiv:2506.21901, 2025.

- [14] Seagate Technology. OCP Composable Memory Appliance (CMA) Base Specification. <https://www.opencompute.org/documents/final-2024-ocp-cma-cfm-base-specification-rev1-1-pdf>.
- [15] M. El-Batal and H. Fan. CXL Composable Memory Appliance (CMA) and Composability Fabric Manager (CFM) Solution Architecture. <https://www.youtube.com/watch?v=jhELiHyr9MU>.
- [16] H. A. Maruf, H. Wang, A. Dhanotia, J. Weiner, N. Agarwal, P. Bhattacharya, C. Petersen, M. Chowdhury, S. Kanaujia, and P. Chauhan. TPP: Transparent page placement for CXL-enabled tiered-memory. In *Proc. ASPLOS*, 2023.
- [17] Y. Sun, Y. Yuan, Z. Yu, R. Kuper, C. Song, J. Huang, H. Ji, S. Agarwal, J. Lou, I. Jeong, R. Wang, J. H. Ahn, T. Xu, and N. S. Kim. Demystifying CXL memory with genuine CXL-ready systems and devices. In *Proc. MICRO*, 2023.
- [18] Z. Wang, S. Mahar, L. Li, J. Park, J. Kim, T. Michailidis, Y. Pan, M. Shen, T. Rosing, D. Tullsen, S. Swanson, and J. Zhao. The Hitchhiker’s Guide to Programming and Optimizing Cache Coherent Heterogeneous Systems: CXL, NVLink-C2C, and AMD Infinity Fabric. *arXiv:2411.02814*, 2024.
- [19] Kubernetes. Dynamic Resource Allocation. <https://kubernetes.io/docs/concepts/scheduling-eviction/dynamic-resource-allocation/>.
- [20] Kubernetes SIG Node. Kubernetes Dynamic Resource Allocation Architecture. Kubernetes, 2024.
- [21] CNCF. Container Device Interface (CDI). <https://github.com/cncf-tags/container-device-interface>.
- [22] The llm-d project. llm-d: Kubernetes-native distributed inference. <https://llm-d.ai>.
- [23] Qwen Team. Qwen2.5 technical report. arXiv preprint arXiv:2412.15115, 2024.
- [24] Intel. Intel Memory Latency Checker (MLC), v3.11b. <https://www.intel.com/content/www/us/en/developer/articles/tool/intelr-memory-latency-checker.html>.
- [25] M. Weisgut, D. Ritter, P. Tözün, L. Benson, and T. Rabl. CXL Memory Performance for In-Memory Data Processing. In *Proc. VLDB*, 2025.
- [26] M. Weisgut, D. Ritter, F. Schmeller, P. Tözün, and T. Rabl. CXL-Bench: Benchmarking Shared CXL Memory Access. In *ADMS @ VLDB*, 2025.
- [27] R. Ma, T. Ma, J. Li, H. Zha, X. Shang, Q. Hu, Z. Liu, X. Yang, T. Ma, and G. Luo. SAC: Disaggregated KV cache system for sparse attention LLMs with CXL. arXiv preprint arXiv:2606.19746, 2026.
- [28] X. Yang, Y. Zhang, H. Chen, F. Li, G. Fan, Y. Kong, B. Wang, J. Fang, Y. Wang, T. Huang, W. Hu, J. Kao, and J. Jiang. Unlocking the Potential of CXL for Disaggregated Memory in Cloud-Native Databases. In *Proc. SIGMOD/PODS*, 2025.
- [29] A. Dragojević, D. Narayanan, O. Hodson, and M. Castro. FaRM: Fast remote memory. In *Proc. USENIX NSDI*, 2014.
- [30] J. Gu, Y. Lee, Y. Zhang, M. Chowdhury, and K. G. Shin. Efficient memory disaggregation with Infiniswap. In *Proc. USENIX NSDI*, 2017.
- [31] Q. Cai, W. Guo, H. Zhang, D. Agrawal, G. Chen, B. C. Ooi, K.-L. Tan, Y. M. Teo, and S. Wang. Efficient distributed memory management with RDMA and caching. *Proc. VLDB Endow.*, vol. 11, no. 11, pp. 1604–1617, 2018.
- [32] H. Li, D. S. Berger, L. Hsu, D. Ernst, P. Zardoshti, S. Novakovic, M. Shah, S. Rajadnya, S. Lee, I. Agarwal, M. D. Hill, M. Fontoura, and R. Bianchini. Pond: CXL-based memory pooling systems for cloud platforms. In *Proc. ASPLOS*, 2023.

- [33] Y. Zhong, F. Kazhamiaka, P. Zardoshti, S. Teng, R. Fonseca, M. D. Hill, and D. S. Berger. Octopus: Enhancing CXL Memory Pods via Sparse Topology. *arXiv:2501.09020*, 2026.
- [34] Seagate Open Source. Seagate GitHub repository. <https://github.com/Seagate>.

Table 6: TTFT of the reusing request (ms),  $n = 60$  per cell (3 repetitions  $\times$  20 sessions). *cross* means the second request is served by the replica on the other node, so only a shared tier can hit.

| Tier | Arm   | Prefix | p50     | p90     | p99     | p99/p50 | Hit% |
|------|-------|--------|---------|---------|---------|---------|------|
| T0   | same  | 2K     | 568.6   | 576.2   | 580.2   | 1.02    | 0.0  |
| T0   | same  | 8K     | 2306.3  | 2386.1  | 2391.6  | 1.04    | 0.0  |
| T0   | same  | 16K    | 5127.6  | 5191.8  | 5205.8  | 1.02    | 0.0  |
| T0   | same  | 32K    | 12388.8 | 12504.8 | 12585.7 | 1.02    | 0.0  |
| T0   | cross | 2K     | 560.7   | 568.7   | 949.7   | 1.69    | 0.0  |
| T0   | cross | 8K     | 2223.6  | 2279.6  | 2284.8  | 1.03    | 0.0  |
| T0   | cross | 16K    | 4895.0  | 4950.5  | 4966.3  | 1.01    | 0.0  |
| T0   | cross | 32K    | 11816.9 | 11924.7 | 11979.6 | 1.01    | 0.0  |
| T1   | same  | 2K     | 86.6    | 87.3    | 88.0    | 1.02    | 98.8 |
| T1   | same  | 8K     | 117.0   | 120.8   | 122.1   | 1.04    | 99.7 |
| T1   | same  | 16K    | 131.8   | 148.6   | 149.8   | 1.14    | 99.9 |
| T1   | same  | 32K    | 189.1   | 191.0   | 192.0   | 1.02    | 99.9 |
| T1   | cross | 2K     | 563.3   | 570.7   | 958.6   | 1.70    | 0.7  |
| T1   | cross | 8K     | 2218.6  | 2287.1  | 2296.7  | 1.04    | 0.3  |
| T1   | cross | 16K    | 4898.4  | 4937.4  | 4959.4  | 1.01    | 0.1  |
| T1   | cross | 32K    | 11845.0 | 11921.1 | 12015.7 | 1.01    | 0.0  |
| T2   | same  | 2K     | 93.9    | 94.9    | 95.5    | 1.02    | 98.8 |
| T2   | same  | 8K     | 138.4   | 142.5   | 143.4   | 1.04    | 99.7 |
| T2   | same  | 16K    | 170.3   | 187.1   | 189.4   | 1.11    | 99.9 |
| T2   | same  | 32K    | 259.9   | 265.0   | 267.7   | 1.03    | 99.9 |
| T2   | cross | 2K     | 566.8   | 575.7   | 954.8   | 1.68    | 0.7  |
| T2   | cross | 8K     | 2224.6  | 2280.9  | 2298.9  | 1.03    | 0.4  |
| T2   | cross | 16K    | 4901.9  | 4944.4  | 4962.8  | 1.01    | 0.2  |
| T2   | cross | 32K    | 11822.3 | 11926.3 | 12023.7 | 1.02    | 0.1  |
| T4   | same  | 2K     | 99.5    | 130.1   | 131.8   | 1.32    | 94.6 |
| T4   | same  | 8K     | 168.4   | 178.8   | 195.9   | 1.16    | 97.9 |
| T4   | same  | 16K    | 216.3   | 234.6   | 239.7   | 1.11    | 98.9 |
| T4   | same  | 32K    | 320.6   | 347.7   | 366.9   | 1.14    | 99.5 |
| T4   | cross | 2K     | 101.6   | 134.7   | 521.2   | 5.13    | 95.4 |
| T4   | cross | 8K     | 175.0   | 196.2   | 199.0   | 1.14    | 98.0 |
| T4   | cross | 16K    | 224.6   | 240.6   | 249.3   | 1.11    | 99.0 |
| T4   | cross | 32K    | 322.6   | 357.4   | 381.4   | 1.18    | 99.5 |

Table 7: The honest cost of the tier when reuse *is* local (same arm): the GPU’s own prefix cache is faster than the shared region, by at most  $1.7\times$ .

| Prefix | T1 p50 (ms) | T4 p50 (ms) | T4/T1        |
|--------|-------------|-------------|--------------|
| 2K     | 86.6        | 99.5        | $1.15\times$ |
| 8K     | 117.0       | 168.4       | $1.44\times$ |
| 16K    | 131.8       | 216.3       | $1.64\times$ |
| 32K    | 189.1       | 320.6       | $1.70\times$ |

Table 8: Reproducibility: per-repetition median TTFT (ms) in the cross arm. *spread* is (max−min)/median. Repetitions ran in different tier orders on different days.

| Tier | Prefix | rep 1   | rep 2   | rep 3   | spread |
|------|--------|---------|---------|---------|--------|
| T0   | 2K     | 559.4   | 560.1   | 566.5   | 1.3%   |
| T0   | 8K     | 2221.0  | 2236.4  | 2255.2  | 1.5%   |
| T0   | 16K    | 4889.9  | 4891.0  | 4900.3  | 0.2%   |
| T0   | 32K    | 11798.4 | 11821.7 | 11826.9 | 0.2%   |
| T1   | 2K     | 562.2   | 564.1   | 565.2   | 0.5%   |
| T1   | 8K     | 2214.8  | 2218.8  | 2219.9  | 0.2%   |
| T1   | 16K    | 4895.7  | 4898.4  | 4904.4  | 0.2%   |
| T1   | 32K    | 11843.9 | 11847.4 | 11852.9 | 0.1%   |
| T2   | 2K     | 564.8   | 566.6   | 570.0   | 0.9%   |
| T2   | 8K     | 2209.9  | 2224.0  | 2228.3  | 0.8%   |
| T2   | 16K    | 4899.2  | 4901.3  | 4903.7  | 0.1%   |
| T2   | 32K    | 11801.3 | 11810.1 | 11840.9 | 0.3%   |
| T4   | 2K     | 101.1   | 101.8   | 101.9   | 0.8%   |
| T4   | 8K     | 155.7   | 173.6   | 178.1   | 12.9%  |
| T4   | 16K    | 223.7   | 224.2   | 227.7   | 1.8%   |
| T4   | 32K    | 315.4   | 317.8   | 325.3   | 3.1%   |

Table 9: Comparison of KV-cache sharing and CXL memory systems. Our work is the first to make a composable CXL region schedulable by Kubernetes DRA.

| System          | Medium     | K8s DRA    | P/D-disagg.    | Real CXL hw? |
|-----------------|------------|------------|----------------|--------------|
| Mooncake [7]    | RDMA       | no         | yes            | —            |
| TraCT [11]      | CXL        | no         | yes            | yes          |
| HyMCache [12]   | CXL        | no         | no             | yes          |
| SAC [27]        | CXL        | no         | no             | yes          |
| Pangaea v2 [10] | CXL        | no (NRI)   | no             | yes          |
| <b>Ours</b>     | <b>CXL</b> | <b>yes</b> | <b>no (v1)</b> | <b>yes</b>   |